College Admission Assistant

1. What Is This Project?

The College Admission Assistant is a smart AI chatbot that helps students get instant, accurate answers about college admissions in India. It uses real data from 350 Indian colleges, AI reasoning, and dedicated tools.

Students can ask questions like:

- "What documents do I need for engineering admission?"
- "What are the deadlines for IIT Bombay?"
- "Am I eligible for NIT with 78%?"

Technology	Role in the Project
Python	All backend logic and server scripts
HTML / CSS / JavaScript	Chat interface visible in the browser
n8n	Orchestration — connects all components together
ChromaDB	Stores and searches 350 college records (Vector DB)
MCP (Model Context Protocol)	Gives the AI three callable tools

2. How Does It Work?

Complete flow — from student question to AI answer:

Step	Component	What Happens
1	Student (Browser)	Types a question in the chat UI
2	script.js	Captures text and sends HTTP POST to n8n webhook
3	n8n Webhook Node	Receives the POST request and passes it to AI Agent
4	AI Agent	Decides which tool to call and queries ChromaDB
5	MCP Server Tools	Returns deadlines, document checklists, or eligibility info
6	ChromaDB (RAG)	Searches 350 college records for relevant context
7	AI Agent	Combines tool result + DB context to write a full reply
8	n8n Respond Node	Sends the formatted JSON response back to the UI
9	script.js	Receives JSON and displays reply as a chat bubble

Summary:

UI → n8n → AI Agent → MCP Tools + ChromaDB → Reply → UI

3. Project Folder Structure

All files are organized under one root folder: college_admission_assistant/

File / Folder	Description
college_dataset_350.csv	Raw data of 350 Indian colleges with full details
requirements.txt	Python libraries needed to run the project
.env	Secret API keys — never share or upload to GitHub
README.md	Quick setup guide for the project
ui/index.html	Webpage layout — header, chat area, input box, send button
ui/style.css	Design and colors — fonts, chat bubbles, page look
ui/script.js	Chat logic — sends messages to n8n, shows AI replies
ui/app.py	Small Python server that opens the chat page in the browser
mcp_server/server.py	Gives AI three tools: deadlines, checklist, eligibility
data_prep/build_vector_db.py	Reads CSV and uploads all records to ChromaDB Cloud
vector_db/chroma.sqlite3	Local copy of the vector database
n8n_workflows/workflow.json	The n8n workflow that connects all components

4. Components Explained

4.1 User Interface (ui/)

The UI is a simple browser-based chat application — it looks and works like a messaging app.

Run with:

```
python ui/app.py
```

This opens http://localhost:8501 automatically in your browser.

File	What It Does
index.html	Defines the layout — header, chat area, input box, send button
style.css	Makes everything look good — colors, fonts, chat bubbles
script.js	Handles sending the message to n8n and displaying the reply
app.py	A small Python web server that opens the chat page in the browser

4.2 MCP Server (mcp_server/server.py)

The MCP (Model Context Protocol) server gives the AI Agent three special tools it can call when needed. These are Python functions the AI runs automatically based on the student's question.

```
python mcp_server/server.py
```

Starts on port 8000 using SSE (Server-Sent Events) transport. Keep this terminal open while using the app. The three tools are described in Section 5.

4.3 Vector Database — RAG (data_prep/ and vector_db/)

RAG = Retrieval-Augmented Generation. The AI does not guess — it searches a real database of college information to answer accurately.

- Reads college_dataset_350.csv (350 Indian colleges with full details)
- Each row is converted into a vector (list of numbers) using all-MiniLM-L6-v2
- All vectors are uploaded to ChromaDB Cloud
- When a student asks a question, n8n searches the DB and feeds relevant info to the AI

```
python data_prep/build_vector_db.py
```

ChromaDB Setting	Value
Host	api.trychroma.com
Database Name	College_admission_db
Collection Name	college_admission_data
Embedding Model	all-MiniLM-L6-v2
Authentication	CHROMA_TOKEN from .env file

4.4 n8n Workflow (n8n_workflows/workflow.json)

n8n is a no-code automation tool. The workflow file defines how all the pieces connect together.

Node	Type	What It Does
Webhook	Trigger	Listens for POST requests from the UI at /admission-chat
AI Agent	Processing	Receives message, calls tools, searches DB, generates reply
Respond to Webhook	Output	Sends the AI reply back to the UI as JSON

AI Agent system prompt:

"You are an expert College Admission Assistant. Use the provided tools and context to answer questions about eligibility, required documents, deadlines, and provide checklists."

5. Tools Provided by the AI

The MCP server gives the AI three callable tools. Each is a Python function the AI calls automatically based on what the student asks.

Tool 1 — get_deadlines

Purpose: Returns admission deadlines for a specific university.

Input: University name (text) → Output: Exam name and counseling dates

University	Exam / Deadline Information
IIT Bombay	JEE Advanced: May | JoSAA Counseling: June–July
IIT Delhi	JEE Advanced: May | JoSAA Counseling: June–July
IIT Madras	JEE Advanced: May | JoSAA Counseling: June–July
BITS Pilani	BITSAT Session 1: May | Session 2: June
NIT Trichy	JEE Main: Jan & April | JoSAA Counseling: June
NIT Surathkal	JEE Main: Jan & April | JoSAA Counseling: June
Delhi University	CUET UG: May | CSAS Portal Allocation: July
Anna University	TNEA Counseling Registration: May
VIT Vellore	VITEEE: April | Counseling: May
SRMIST	SRMJEEE: April & June
IISc Bangalore	IISc Portal Registration: June
Jadavpur University	WBJEE: April | Counseling: July
AIIMS Delhi	NEET UG: May | MCC Counseling: July

Tool 2 — generate_checklist

Purpose: Returns the list of documents needed for admission.

Input: Course type — engineering, medical, or degree

Course Type	Documents Required
engineering	12th Marksheet, JEE Scorecard, Aadhar Card, Category Certificate
medical	12th PCB Marksheet, NEET Scorecard, Medical Certificate
degree	12th Marksheet, CUET Scorecard, Transfer Certificate, Photos

Tool 3 — check_eligibility

Purpose: Tells a student which colleges they can apply to based on their 12th percentage.

Input: 12th percentage (a number)

Percentage	Eligibility Result
90% and above	Top-tier colleges possible — IITs, IISc, AIIMS
75% to 89%	Good for NITs and BITS Pilani

60% to 74%	Eligible for many private colleges
Below 60%	Consider diploma course or improvement exam

Example usage:

Student asks: "Am I eligible for NIT with 78%?"

AI calls: check_eligibility(78) → Returns: "Good for NITs / BITS"

6. Setup Instructions

Prerequisites: Python 3.9+, pip, n8n (local or server), ChromaDB Cloud account & token, LLM API key (OpenAI or Gemini).

Step	Action	Command / Notes
1	Install Python Libraries	pip install -r requirements.txt
2	Set Up .env File	Add: CHROMA_TOKEN=your_token_here (never share publicly)
3	Build Vector Database	python data_prep/build_vector_db.py (run only once)
4	Start MCP Server	python mcp_server/server.py → keeps running on port 8000
5	Set Up n8n Workflow	Import workflow.json → Add LLM key → Click Activate
6	Link MCP Node in n8n	Set MCP Tool node URL to: http://localhost:8000/sse
7	Link Vector Store in n8n	Connect to your ChromaDB Cloud database
8	Update Webhook URL in UI	Open ui/script.js → paste the webhook URL from n8n
9	Start the UI	python ui/app.py → opens http://localhost:8501

7. Dependencies

Library	What It Does
mcp	Runs the MCP server and defines AI-callable tools
chromadb	Connects to ChromaDB Cloud vector database
sentence-transformers	Converts text into vectors for semantic search
requests	Sends HTTP requests from the UI to n8n
pydantic	Validates data types and structures
streamlit	Optional alternate UI (may be used)
pandas	Reads the CSV file containing college data
python-dotenv	Loads secret keys from the .env file

8. Environment Variables

The .env file stores secret configuration that must never be shared publicly or committed to GitHub. The .gitignore file already excludes it automatically.

Variable	Description
CHROMA_TOKEN	Your ChromaDB Cloud API token — required for database authentication

Add this to your .env file in the project root:

```
CHROMA_TOKEN=your_chromadb_cloud_token_here
```

9. How Data Flows End to End

Example student message: "What documents do I need for medical admission?"

#	Component	Action Taken
1	Student (Browser)	Types the question into the chat UI
2	script.js	Sends HTTP POST → http://localhost:5678/webhook/admission-chat
3	n8n Webhook Node	Receives and passes the POST request to the AI Agent
4	n8n AI Agent	Identifies intent and calls: generate_checklist("medical")
5	MCP Server	Returns: 12th PCB Marksheet, NEET Scorecard, Medical Certificate
6	ChromaDB (RAG)	Searched for relevant medical college admission context
7	AI Agent	Combines tool result + DB context → writes a complete reply
8	n8n Respond Node	Sends formatted JSON response back to the UI
9	script.js	Receives JSON and renders the reply as a chat bubble
10	Student	Reads the answer in the browser — full response displayed

10. Troubleshooting Common Problems

Problem	Likely Cause	Fix
Browser shows blank page	UI server not running	Run: python ui/app.py
No response in chat	n8n workflow not activated	Open n8n → click Activate
"No deadline info for X"	University not in supported list	Check Section 5 for valid names
Vector DB build fails	Wrong CSV file path	Update file path in build_vector_db.py
MCP server won't start	Port 8000 already in use	Stop other process or change port
CHROMA_TOKEN error	.env file missing or wrong token	Check .env and verify ChromaDB token
n8n can't reach MCP	MCP server not running	Run mcp_server/server.py first
pandas not found	Library not installed	Run: pip install pandas

Quick Reference — Commands Summary

Run all commands from the project root folder. Open separate terminal windows for servers.

Purpose	Command
Install all dependencies	pip install -r requirements.txt
Build vector database (run once)	python data_prep/build_vector_db.py
Start MCP Server — Terminal 1	python mcp_server/server.py
Start UI Server — Terminal 2	python ui/app.py
Open chat in browser	http://localhost:8501

Startup order: MCP Server → n8n Workflow (Activate) → UI Server → Open Browser